Language Technology

Chapter 10: Word Sequences

https://link.springer.com/chapter/10.1007/978-3-031-57549-5_10

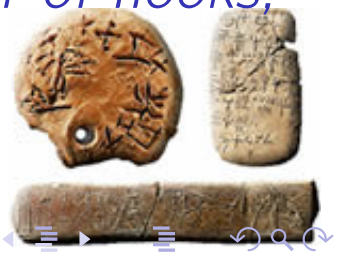
Pierre Nugues

Pierre.Nugues@cs.lth.se

September 7 and 10, 2026

Word Sequences

Words occur in specific contexts of use.
Pairs such as *strong* and *tea*, or *powerful* and *computer*, are not random associations.
Psycholinguistic evidence shows that it is difficult to distinguish *writer* from *rider* without context.
A listener will discard the improbable sequence *rider of books* and prefer *writer of books*.
A language model is a statistical estimate of the probability of a word sequence.
Originally developed for speech recognition, the language-model component predicts the next word given a sequence of preceding words: *the writer of books, novels, poetry*, etc., rather than *the writer of hooks, nobles, poultry*, ...



Statistical Estimates

We build a model to estimate the likelihood of competing sequence hypotheses, for example:

$$P(\text{I wanted to be a book writer})$$

and

$$P(\text{I wanted to be a book rider})$$

and retain the higher probability.
The same principle applies to:

$$P(\text{The buoys eat the sand which is})$$

versus

$$P(\text{The boys eat the sandwiches}).$$

N-Grams

Types are the distinct words of a text; **tokens** are all the word occurrences (or symbols).
The slogans from *Nineteen Eighty-Four*

> *War is peace*
> *Freedom is slavery*
> *Ignorance is strength*

contain 9 tokens and 7 types.

- Unigrams are single words
- Bigrams are sequences of two words
- Trigrams are sequences of three words

Trigrams

Word	Rank	More likely alternatives
We	9	*The This One Two A Three Please In*
need	7	*are will the would also do*
to	1	
resolve	85	*have know do. . .*
all	9	*the this these problems. . .*
of	2	*the*
the	1	
important	657	*document question first. . .*
issues	14	*thing point to. . .*
within	74	*to of and in that. . .*
the	1	
next	2	*company*
two	5	*page exhibit meeting day*
days	5	*weeks years pages months*

Probabilistic Models of a Word Sequence

$$
\begin{aligned}
P(S) &= P(w_1, \ldots, w_n) \\
&= P(w_1)\, P(w_2 \mid w_1)\, P(w_3 \mid w_1, w_2) \ldots P(w_n \mid w_1, \ldots, w_{n-1}) \\
&= \prod_{i=1}^{n} P(w_i \mid w_1, \ldots, w_{i-1}).
\end{aligned}
$$

The probability $P(\text{It was a bright cold day in April})$ from *Nineteen Eighty-Four* corresponds to the probability of *It* beginning the sentence, then *was* given *It*, then *a* given *It was*, and so on until the end of the sentence:

$$
\begin{aligned}
P(S) = P(\text{It}) &\times P(\text{was} \mid \text{It}) \times P(\text{a} \mid \text{It, was}) \\
&\times P(\text{bright} \mid \text{It, was, a}) \times \ldots \\
&\times P(\text{April} \mid \text{It, was, a, bright, } \ldots, \text{in}).
\end{aligned}
$$

Approximations

Bigram approximation:

$$P(w_i \mid w_1, w_2, \ldots, w_{i-1}) \approx P(w_i \mid w_{i-1}).$$

Trigram approximation:

$$P(w_i \mid w_1, w_2, \ldots, w_{i-1}) \approx P(w_i \mid w_{i-2}, w_{i-1}).$$

Using a trigram language model, $P(S)$ is approximated as:

$$\begin{aligned} P(S) \approx & P(\text{It}) \times P(\text{was} \mid \text{It}) \times P(\text{a} \mid \text{It, was}) \\ & \times P(\text{bright} \mid \text{was, a}) \times \ldots \\ & \times P(\text{April} \mid \text{day, in}). \end{aligned}$$

Counting Bigrams with Unix Tools

1. `tr -cs 'A-Za-z' '\n' < input_file > token_file`
   Tokenizes the input and creates a file containing the unigrams (one token per line).
2. `tail +2 < token_file > next_token_file`
   Creates a second unigram file that starts at the second word of the first file.
3. `paste token_file next_token_file > bigrams`
   Merges the two files line by line. Each line of `bigrams` contains the words at positions i and $i+1$ separated by a tab.
4. The bigrams can then be counted with the same pipeline used for unigrams.

Counting Bigrams in Python

```
bigrams = [tuple(words[inx:inx + 2])
           for inx in range(len(words) - 1)]
```

The remainder of the counting function is almost identical to the unigram version:

```
def count_bigrams(words):
    bigrams = [tuple(words[inx:inx + 2])
               for inx in range(len(words) - 1)]
    frequencies = {}
    for bigram in bigrams:
        if bigram in frequencies:
            frequencies[bigram] += 1
        else:
            frequencies[bigram] = 1
    return frequencies
```

Maximum Likelihood Estimate

Bigrams:

$$P_{\text{MLE}}(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i)}{\sum_w C(w_{i-1}, w)} = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}.$$

Trigrams:

$$P_{\text{MLE}}(w_i \mid w_{i-2}, w_{i-1}) = \frac{C(w_{i-2}, w_{i-1}, w_i)}{C(w_{i-2}, w_{i-1})}.$$

Conditional Probabilities

A common mistake when computing the conditional probability $P(w_i \mid w_{i-1})$ is to use

$$\frac{C(w_{i-1}, w_i)}{\#\text{bigrams}}.$$

This formula actually estimates the joint probability $P(w_{i-1}, w_i)$.
The correct maximum-likelihood estimate is

$$P_{\text{MLE}}(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}.$$

Proof:

$$P(w_1, w_2) = P(w_1)\,P(w_2 \mid w_1) = \frac{C(w_1)}{\#\text{words}} \times \frac{C(w_1, w_2)}{C(w_1)} = \frac{C(w_1, w_2)}{\#\text{words}}.$$

Demo

https://github.com/pnugues/pnlp/tree/main/notebooks

Training an *N*-gram Model

- The model is trained on a portion of the corpus called the **training set**.
- It is evaluated on a different portion called the **test set**.
- The vocabulary may be derived from the corpus (e.g. the 20 000 most frequent words) or taken from an external lexicon.
- The vocabulary can be closed or open.
- A closed vocabulary admits no new words.
- An open vocabulary maps unseen words (in either the training or the test set) to a special symbol such as <UNK>.

Probability of a Sentence: Unigrams

<s> *A good deal of the literature of the past was, indeed, already being transformed in this way* </s>

w_i	$C(w_i)$	#words	$P_{\text{MLE}}(w_i)$
<s>	7072	–	–
a	2482	108140	0.023
good	53	108140	0.00049
deal	5	108140	4.62×10^{-5}
of	3310	108140	0.031
the	6248	108140	0.058
literature	7	108140	6.47×10^{-5}
of	3310	108140	0.031
the	6248	108140	0.058
past	99	108140	0.00092
was	2211	108140	0.020
indeed	17	108140	0.00016
already	64	108140	0.00059
being	80	108140	0.00074
transformed	1	108140	9.25×10^{-6}
in	1759	108140	0.016
this	264	108140	0.0024
way	122	108140	0.0011
</s>	7072	108140	0.065

Probability of a Sentence: Bigrams

<s> A good deal of the literature of the past was, indeed, already being transformed in this way </s>

w_{i-1}, w_i	$C(w_{i-1}, w_i)$	$C(w_{i-1})$	$P_{\text{MLE}}(w_i \mid w_{i-1})$
<s> *a*	133	7072	0.019
a good	14	2482	0.006
good deal	0	53	0.0
deal of	1	5	0.2
of the	742	3310	0.224
the literature	1	6248	0.0002
literature of	3	7	0.429
of the	742	3310	0.224
the past	70	6248	0.011
past was	4	99	0.040
was indeed	0	2211	0.0
indeed already	0	17	0.0
already being	0	64	0.0
being transformed	0	80	0.0
transformed in	0	1	0.0
in this	14	1759	0.008
this way	3	264	0.011
way </s>	18	122	0.148

Sparse Data

Given a vocabulary of 20 000 types, the number of possible bigrams is $20\,000^2 = 400\,000\,000$.
For trigrams the figure rises to $20\,000^3 = 8\,000\,000\,000\,000$.
Common smoothing methods include:

- Laplace (add-one) smoothing
- Linear interpolation:

$$\begin{aligned} P_{\text{DelInterpolation}}(w_n \mid w_{n-2}, w_{n-1}) = {} & \lambda_1 P_{\text{MLE}}(w_n \mid w_{n-2}, w_{n-1}) \\ & + \lambda_2 P_{\text{MLE}}(w_n \mid w_{n-1}) \\ & + \lambda_3 P_{\text{MLE}}(w_n) \end{aligned}$$

- Good–Turing estimation
- Back-off
- Kneser–Ney smoothing

Laplace's Rule

$$P_{\text{Laplace}}(w_{i+1} \mid w_i) = \frac{C(w_i, w_{i+1}) + 1}{C(w_i) + |V|}.$$

w_i, w_{i+1}	$C(w_i, w_{i+1}) + 1$	$C(w_i) + \lvert V \rvert$	$P_{\text{Lap}}(w_{i+1} \mid w_i)$
<s> *a*	134	7072+8635	0.0085
a good	15	2482+8635	0.0013
good deal	1	53+8635	0.00012
deal of	2	5+8635	0.00023
of the	743	3310+8635	0.062
the literature	2	6248+8635	0.00013
literature of	4	7+8635	0.00046
of the	743	3310+8635	0.062
the past	71	6248+8635	0.0048
past was	5	99+8635	0.00057
was indeed	1	2211+8635	0.000092
indeed already	1	17+8635	0.00012
already being	1	64+8635	0.00011
being transformed	1	80+8635	0.00011
transformed in	1	1+8635	0.00012
in this	15	1759+8635	0.0014
this way	4	264+8635	0.00045
way </s>	19	122+8635	0.0022

Good–Turing

Laplace's rule shifts a large amount of probability mass to very unlikely bigrams. Good–Turing estimation is more effective.
Let N_c be the number of n-grams that occur exactly c times in the corpus.
N_0 is the number of unseen n-grams, N_1 the number seen once, N_2 the number seen twice, etc.
The frequency of n-grams that occur c times is re-estimated as:

- Unseen n-grams: $c^* = \frac{N_1}{N_0}$
- n-grams seen once: $c^* = \frac{2N_2}{N_1}$

More generally:

$$c^* = (c+1)\frac{E(N_{c+1})}{E(N_c)}.$$

Good–Turing for *Nineteen Eighty-Four*

Nineteen Eighty-Four contains 37 365 unique bigrams and 5 820 bigrams seen twice.
Its vocabulary of 8 635 words generates $8635^2 = 74\,563\,225$ possible bigrams, of which 74 513 701 are unseen.
Re-estimated counts:

- Unseen bigrams: $\frac{37\,365}{74\,513\,701} \approx 0.0005$
- Unique bigrams: $2 \times \frac{5820}{37\,365} \approx 0.31$

Freq.	N_c	c^*	**Freq.**	N_c	c^*
0	74 513 701	0.0005	5	719	3.91
1	37 365	0.31	6	468	4.94
2	5 820	1.09	7	330	6.06
3	2 111	2.02	8	250	6.44
4	1 067	3.37	9	179	8.93

Backoff

If a bigram has never been observed, fall back to the unigram probability:

$$P_{\text{Backoff}}(w_i \mid w_{i-1}) = \begin{cases} \tilde{P}(w_i \mid w_{i-1}) & \text{if } C(w_{i-1}, w_i) \neq 0, \\ \alpha\, P(w_i) & \text{otherwise.} \end{cases}$$

A simplified (non-normalized) version is:

$$P_{\text{Backoff}}(w_i \mid w_{i-1}) = \begin{cases} \dfrac{C(w_{i-1}, w_i)}{C(w_{i-1})} & \text{if } C(w_{i-1}, w_i) \neq 0, \\ \dfrac{C(w_i)}{\#\text{words}} & \text{otherwise.} \end{cases}$$

Note that the probabilities no longer sum to one.

Backoff: Example

w_{i-1}, w_i	$C(w_{i-1}, w_i)$		$C(w_i)$	$P_{\text{Backoff}}(w_i \mid w_{i-1})$
<s> *a*	133		2482	0.019
a good	14		53	0.006
good deal	0	backoff	5	4.62×10^{-5}
deal of	1		3310	0.2
of the	742		6248	0.224
the literature	1		7	0.00016
literature of	3		3310	0.429
of the	742		6248	0.224
the past	70		99	0.011
past was	4		2211	0.040
was indeed	0	backoff	17	0.00016
indeed already	0	backoff	64	0.00059
already being	0	backoff	80	0.00074
being transformed	0	backoff	1	9.25×10^{-6}
transformed in	0	backoff	1759	0.016
in this	14		264	0.008
this way	3		122	0.011
way </s>	18		7072	0.148

These figures are not true probabilities. Discounting the observed bigrams (e.g. with Good–Turing) and then scaling the unigram probabilities yields Katz back-off.

Quality of a Language Model (I)

The quality of a language model is measured by how accurately it predicts word sequences $P(w_1, \ldots, w_n)$: the higher the probability, the better.
The model is estimated on a training set and evaluated on a held-out test set.
With the usual n-gram approximations we have:

$$P(w_1, \ldots, w_n) = \prod_{i=1}^{n} P(w_i) \quad \text{(unigrams)}$$

$$P(w_1, \ldots, w_n) = P(w_1) \prod_{i=2}^{n} P(w_i \mid w_{i-1}) \quad \text{(bigrams)}$$

$$P(w_1, \ldots, w_n) = P(w_1) P(w_2 \mid w_1) \prod_{i=3}^{n} P(w_i \mid w_{i-2}, w_{i-1}) \quad \text{(trigrams)}$$

Quality of a Language Model (II)

Because the probability depends on the length of the sequence, we use the geometric mean to obtain a length-normalized measure:

$$\text{Unigrams:} \quad \sqrt[n]{\prod_{i=1}^{n} P(w_i)}$$

$$\text{Bigrams:} \quad \sqrt[n]{P(w_1)\prod_{i=2}^{n} P(w_i \mid w_{i-1})}$$

In practice we work with logarithms and compute the average log-probability per word:

$$\text{Unigrams:} \quad \frac{1}{n}\sum_{i=1}^{n} \log_2 P(w_i)$$

$$\text{Bigrams:} \quad \frac{1}{n}\left(\log_2 P(w_1) + \sum_{i=2}^{n} \log_2 P(w_i \mid w_{i-1})\right)$$

Perplexity

Mean sequence probabilities are usually very small and hard to interpret. The conventional metric is therefore **perplexity**, the inverse of the geometric mean of the sequence probability:

$$\begin{aligned} \mathrm{PP} &= \frac{1}{\sqrt[n]{P(w_1, w_2, \ldots, w_n)}} \\ &= \frac{1}{\sqrt[n]{\prod_{i=1}^{n} P(w_i \mid w_1, \ldots, w_{i-1})}}. \end{aligned}$$

Lower perplexity is better.
In practice we compute the entropy rate

$$H(L) = -\frac{1}{n} \log_2 P(w_1, \ldots, w_n)$$

and obtain perplexity by

$$\mathrm{PP} = 2^{H(L)}.$$

Mathematical Background

Entropy rate:

$$H_{\text{rate}} = -\frac{1}{n} \sum_{w_1,\ldots,w_n \in L} P(w_1,\ldots,w_n) \log_2 P(w_1,\ldots,w_n).$$

Cross-entropy:

$$H(P,M) = -\frac{1}{n} \sum_{w_1,\ldots,w_n \in L} P(w_1,\ldots,w_n) \log_2 M(w_1,\ldots,w_n).$$

Under suitable assumptions this reduces to

$$H(P,M) = \lim_{n\to\infty} -\frac{1}{n} \log_2 M(w_1,\ldots,w_n).$$

We evaluate the cross-entropy of a test sequence (governed by the true distribution P) under a bigram or trigram model M estimated on a training set.

Masked Language Models

The language models seen so far are **causal** (or **autoregressive**):

$$\arg\max_{x_i \in V} P(x_i \mid x_1, x_2, \ldots, x_{i-1}).$$

Masked language models predict a word from both left and right context, for example:

> *A good deal of the literature of the [MASK] was indeed already being transformed in this way*

given the original sentence

> *A good deal of the literature of the* **past** *was indeed already being transformed in this way.*

They correspond to cloze tests used in language learning:

$$\arg\max_{x_i \in V} P(x_i \mid x_1, \ldots, x_{i-1}, x_{i+1}, \ldots, x_n).$$

High-quality masked models require large neural architectures; Transformers are a prominent example.

Generating Text with a Language Model

In speech recognition a language model helps predict the next word x_i given the preceding words and the acoustic signal A:

$$\arg\max_{x_i \in V} P(x_i \mid x_1, \ldots, x_{i-1}, A).$$

Removing the acoustic input yields the pure language-model distribution

$$P(x_i \mid x_1, \ldots, x_{i-1}).$$

The predicted word can be appended to the sequence and the process repeated, thereby generating text.

Generating Text with Bigrams

For simplicity we use bigrams:

$$P(x_i \mid x_{i-1}).$$

Starting from a seed word such as *Hector*, we sample the next word according to the empirical distribution

$$P(x \mid \text{hector}).$$

```
[(('hector', 'and'), 0.1167),
 (('hector', 'son'), 0.0521),
 (('hector', 's'),   0.0479),
 (('hector', 'was'), 0.0313),
 (('hector', 'in'),  0.0229),
 ...]
```

Drawing the Next Word

We draw the next word with `np.random.multinomial()`, which returns a one-hot vector according to the supplied distribution:

```
np.random.multinomial(1, [0.3, 0.5, 0.2])
```

Repeated draws produce vectors such as

```
[0 0 1]
[0 1 0]
[0 1 0]
...
```

In the long run the frequencies of the three outcomes approach 30 %, 50 % and 20 % respectively.

Generating Text

We begin with a seed word (e.g. *Hector*), sample the next word from the multinomial distribution, and iterate.

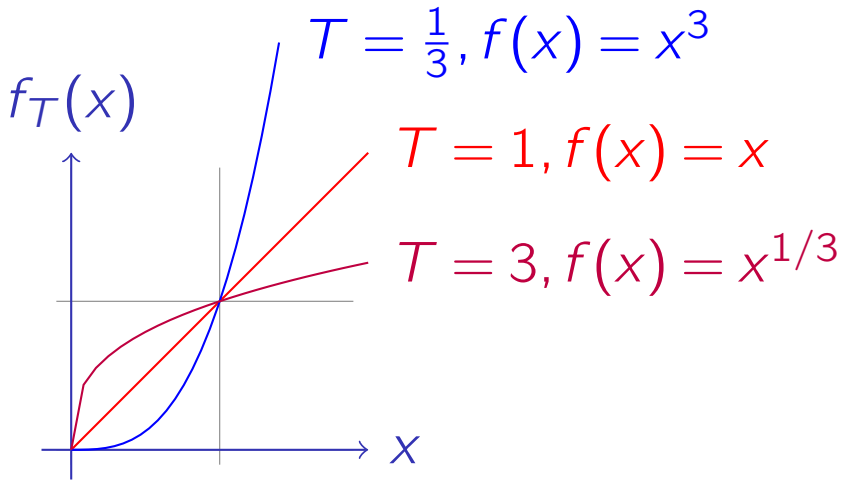
A typical generated fragment looks like:

> *hector has slain noble agenor away from your prophets the bravest of the trojans while achilles was choking him roughly away and as all alone for evermore hades that is not fallen. . .*

Transforming the Distribution

The distribution over the second word of a bigram can be sharpened or flattened by a temperature parameter.
Following Chollet (*Deep Learning with Python*, 2nd ed., pp. 369 & 373) we apply the transformation

$$f_T(x) = x^{1/T}.$$



Code

```
def power_transform(distribution, T=0.5):
    new_dist = np.power(distribution, 1/T)
    return new_dist / np.sum(new_dist)
```

Demo: https://github.com/pnugues/pnlp/tree/main/notebooks

Results

With temperature $T = 3.0$:

hector fearless of perimedes leader of saturn devise evil for juno if ever whereas in that forms on him spear or be from over land priam would prove me honour

With temperature $T = 0.5$:

hector and he was lying dream went up to the achaeans will be bought nor of the shield of his father jove in the trojans and the son of the achaeans and the argives and the trojans and the body of the achaeans if you are you have been dearest to

Other Statistical Formulas

- Mutual information (strength of association):

$$I(w_i, w_j) = \log_2 \frac{P(w_i, w_j)}{P(w_i)P(w_j)} \approx \log_2 \frac{N \cdot C(w_i, w_j)}{C(w_i)C(w_j)}.$$

- t-score (confidence of association):

$$t(w_i, w_j) \approx \frac{C(w_i, w_j) - \frac{1}{N} C(w_i)C(w_j)}{\sqrt{C(w_i, w_j)}}.$$

t-Scores with the Word *set*

Word	Frequency	Bigram set+word	t-score
up	134 882	5512	67.980
a	1 228 514	7296	35.839
to	1 375 856	7688	33.592
off	52 036	888	23.780
out	123 831	1252	23.320

Source: Bank of English

Mutual Information with the Word *surgery*

Word	Frequency	Bigram word+surgery	Mutual info
arthroscopic	3	3	11.822
pioneering	3	3	11.822
reconstructive	14	11	11.474
refractive	6	4	11.237
rhinoplasty	5	3	11.085

Source: Bank of English

Mutual Information in Python

```
def mutual_info(words, freq_unigrams, freq_bigrams):
    mi = {}
    factor = len(words) * len(words) / (len(words) - 1)
    for bigram in freq_bigrams:
        mi[bigram] = math.log(
            factor * freq_bigrams[bigram] /
            (freq_unigrams[bigram[0]] *
             freq_unigrams[bigram[1]]), 2)
    return mi
```

t-Scores in Python

```
def t_scores(words, freq_unigrams, freq_bigrams):
    ts = {}
    for bigram in freq_bigrams:
        ts[bigram] = (
            (freq_bigrams[bigram] -
             freq_unigrams[bigram[0]] *
             freq_unigrams[bigram[1]] / len(words)) /
            math.sqrt(freq_bigrams[bigram]))
    return ts
```